● HANDS-ON WORKSHOP · 90 MINUTES

Building Production APIs with TypeScript

From zero to deployed — type-safe routes, database integration, auth middleware, and CI/CD. Everything you need to ship confidently.

Devon Hartley
Staff Engineer @ Vercel · @devonhartley

Prerequisites: Node 20+, Docker, VS Code
Repo: github.com/devon/ts-api-workshop

What we'll build today.

0:00 – 0:15

Project setup & scaffolding

Initialize a Hono app with TypeScript strict mode, configure ESLint, Prettier, and path aliases.

0:15 – 0:35

Type-safe routes with Zod

Define request/response schemas, build validated endpoints, and generate OpenAPI docs automatically.

0:35 – 0:55

Database with Drizzle ORM

Connect Postgres via Docker, define schemas, run migrations, and write type-safe queries.

0:55 – 1:15

Auth middleware & testing

JWT auth middleware, role-based guards, integration tests with Vitest, and test fixtures.

1:15 – 1:30

Deploy to production

Dockerfile, GitHub Actions CI, deploy to Fly.io, health checks, and monitoring basics.

The type system is your runtime guard.

Define your API contract once with Zod schemas. Get compile-time safety, runtime validation, and auto-generated documentation — from a single source of truth.

01 **Compile-time errors, not runtime crashes**
TypeScript catches schema mismatches before you deploy

02 **Automatic request validation**
Zod parses and validates every incoming request body

03 **OpenAPI docs for free**
Generate Swagger UI from your Zod schemas — always in sync

routes/users.ts

```
import { z } from 'zod'
import { Hono } from 'hono'
import { zValidator } from '@hono/zod-validator'

// Define the contract once
const CreateUser = z.object({
name: z.string().min(1),
email: z.string().email(),
role: z.enum(['admin', 'user']),
})

// Route is fully typed + validated
app.post(
'/users',
zValidator('json', CreateUser),
async (c) => {
// body is typed as CreateUser
const body = c.req.valid('json')
const user = await db.insert(users)
.values(body).returning()
return c.json(user, 201)
}
)
```

EXERCISE 2 · 10 MINUTES

Build a CRUD API for a todo list.

YOUR TASKS

1. Define a `TodoSchema` with Zod (title, completed, priority)
2. Create `GET /todos` with optional `?completed=true` filter
3. Create `POST /todos` with validated JSON body
4. Create `PATCH /todos/:id` with partial update schema

★ Bonus: Add pagination with `?page=1&limit=10`

HINTS

```
// Partial schemas for PATCH:
CreateTodo.partial()

// Query param validation:
zValidator('query', schema)

// Coerce query strings:
z.coerce.number().default(1)

// Check the solution:
git checkout solution/ex-2
```

What changes.

BEFORE · UNTYPED EXPRESS

- ✕ Manual `req.body` parsing — crashes on malformed JSON
- ✕ API docs written by hand — stale within a week
- ✕ Type errors discovered in production at 3am
- ✕ Frontend and backend types drift silently
- ✕ 400+ lines of validation boilerplate per route

AFTER · HONO + ZOD + DRIZZLE

- ✓ Auto-validated requests — invalid input returns 400 with details
- ✓ OpenAPI spec generated from schemas — always accurate
- ✓ Type errors caught at compile time — before code review
- ✓ Shared Zod schemas between frontend and backend
- ✓ 5 lines of schema = validation + types + docs

67% fewer production type-related incidents in teams that adopted this stack (Source: 2025 State of TypeScript Report)

01

Define your contract once — in Zod — and let everything else derive from it.

Types, validation, documentation, and client SDK generation. Single source of truth eliminates drift.

02

Test the integration, not the units. Hit a real database in tests.

Docker Compose + test fixtures + transactions = fast, reliable, realistic tests that catch real bugs.

03

Ship on day one. Iterate from production, not localhost.

A deployed "hello world" with CI/CD is worth more than a perfect local setup. Get the pipeline working first.

Thanks for building with me today.

WORKSHOP REPO

github.com/devon/ts-api-workshop

SLIDES & NOTES

devon.dev/talks/ts-api-workshop

QUESTIONS?

Discord: devon.dev/chat
Twitter: @devonhartley

FURTHER READING hono.dev/docs · zod.dev · orm.drizzle.team · fly.io/docs